Group Project Final Presentation and Demonstration Guidelines

PR3: Final Demonstration | PR4: System Presentation

Purpose

Each group must prepare a professional and technically sound slide presentation that demonstrates the software engineering process, technical implementation, DevOps practices, system design, and working system functionality. The presentation must use proper software engineering terminology, standard modelling notation, and clear technical explanation suitable for Software Engineering students.

Expected Slide Structure

No.	Component	Main Evidence Expected
1	Team Profile and Roles	Member photos, roles, contributions, and evidence of task distribution
2	Project Overview	Problem, users, objectives Use case diagram / user stories
3	DevOps Practices and Project Management	Git/Github, CI/CD, deployment, tools, and collaboration evidence
4	System Architecture and Design	Standard UML/system diagrams such as architecture, sequence, activity, and deployment diagrams
5	Database and Data Management	Data model, main entities/tables, validation, and realistic demonstration data
6	System/Application Demonstration	End-to-end working processes with meaningful data and real system interaction

1. Team Profile and Roles

Introduce all team members and clearly state their project roles, responsibilities, and actual contributions.

- Name and photo of each team member.
- Assigned role
- Brief explanation of each member's contribution throughout the project.
- Evidence of teamwork and task distribution, such as sprint tasks, GitHub commits, Jira assignments, testing responsibilities, or documentation contributions.

2. Project Overview and Evolution

Describe the project by summarising the original proposal (and enhancement, if any)

- Project title and problem statement.
- Target users, stakeholders, and context of use.
- Project objectives and intended benefits.
- Main features and functional scope.
- Enhancements or changes made during the project.

3. DevOps Practices and Project Management

Explain how the team applied DevOps practices that combine software development and operations. The explanation must be supported with evidence from the tools used during the project.

3.1 Version Control and Collaboration

- GitHub repository structure.
- Branching strategy, such as main, development, feature branches, release branches, or hotfix branches.
- Pull request, merge, or code review process.
- Commit history and contribution evidence.
- Conflict resolution or collaboration practices, if applicable.

3.2 CI/CD and Deployment

- Continuous Integration practices, such as automated build, automated testing, linting, or code validation.
- Continuous Delivery or deployment process.
- Deployment environment, such as Firebase, Vercel, Netlify, Render, shared hosting, cloud server, or local server.
- Build and release process.
- Any automation used in deployment, or version control.

4. System Architecture and Design

Present the technical design of the system using standard Software Engineering diagrams and notation. All diagrams must be readable, properly labelled, and consistent with the actual implemented system.

4.1 System Architecture Diagram

- Client application, such as web, mobile, or desktop interface.
- Server-side application, API, or backend services.
- Database or persistent storage.
- External services or third-party APIs.
- Authentication or authorization service.
- Deployment or hosting environment.

4.2 Sequence Diagram

Use UML sequence diagram notation to show interaction between users, frontend, backend/API, database, and external services for at least one major system process.

Examples:

- Booking, order, payment, submission, approval, or request process.
- Search and filtering process.
- Report or dashboard generation process.
- Integration with an external API or service.

4.3 Activity Diagram

- Start and end nodes.
- User actions and system actions.
- Decision points and alternative flows.
- Parallel activities, where applicable.
- Clear representation of the business workflow.

4.4 Deployment Diagram

- Client device or browser.
- Web server or application server.
- Database server or cloud database.
- Cloud service, hosting platform, or local server.
- API services and external integrations.
- Network or communication protocol, where relevant.

5. Database and Data Management

Describe how the system stores, processes, retrieves, validates, and protects data.

- Database or storage technology used.
- Main tables, collections, entities, or data files.
- Entity Relationship Diagram, class diagram, or data model, where appropriate.
- Data validation and data integrity considerations.
- Security considerations for sensitive data, such as password handling, user roles, access control, or data privacy.

Data requirement
The system must contain sufficient and meaningful data for demonstration. The demo should not rely only on empty screens, hardcoded values, or unrealistic dummy records.

6. System/Application Demonstration

The demonstration must show the complete end-to-end process of the system. Students must demonstrate actual system processing and data flow, not only static user interface screens.

- User authentication or access flow, if applicable.
- Main user workflows from beginning to end.
- Create, read, update, and delete operations, if relevant.
- Search, filter, sort, or recommendation functions.
- Dashboard, reporting, analytics, or summary features.
- Error handling, validation, and exception handling.
- Integration with external APIs or services, if applicable.
- Interaction between frontend, backend, database, and external services, where applicable.

Important demo requirement
Prepare enough realistic data to properly demonstrate features such as search, reporting, dashboard, analytics, booking, payment, user management, or approval workflow. The demo must not show only dummy data, incomplete forms, static pages, or hardcoded output.

7. General Presentation Expectations

- Maximum slides = 20
- Use clear, professional, and readable slide design.
- Apply correct Software Engineering terminology.
- Use standard UML notation for diagrams.

- Provide technical explanation, not only general description.
- Show real evidence of development work.
- Demonstrate a working system with meaningful data.
- Ensure all group members participate in the presentation and demonstration.
- Manage presentation and demonstration time effectively.
- Ensure diagrams, screenshots, and demo data are consistent with the implemented system.

Overall expectation

The final presentation and demonstration should prove that the team has not only developed a working application, but has also applied proper Software Engineering practices in requirements analysis, design, implementation, testing, DevOps, deployment, and project management.